

# RELATÓRIO TÉCNICO FINAL

## Desenvolvimento de Sistema de Inteligência Artificial para Automação de Processos Administrativos da UFOP (Projeto IA UFOP)

*Iniciação Científica*

**Discente:** Tchairô Niddan Ruiz Gimenez Leão

**Orientador:** Thiago Fontes Santos

**Instituição:** Universidade Federal de Ouro Preto (UFOP)

**Período de referência:** Fevereiro de 2026 – Agosto de 2026

**Status do projeto:** Encerrado

## Sumário

0. Nota Introdutória
1. Contexto e Evolução do Escopo do Projeto
2. Ambientes Computacionais Utilizados
3. Etapa 1 — Coleta de Dados
4. Etapa 2 — Construção da Base de Conhecimento Vetorial (RAG)
5. Etapa 3 — Seleção de Arquitetura e Ajuste Fino do Modelo
6. Implantação Local e Testes de Hardware
7. Estratégia de Implementação em Hardware Limitado (GGUF / llama.cpp)
8. Elaboração do Artigo Científico
9. Síntese Consolidada de Falhas e Obstáculos Técnicos
10. Síntese Consolidada de Conquistas
11. Estado Final do Projeto e Recomendações para Eventual Continuidade

## Nota Introdutória

Este relatório consolida, de forma fidedigna e detalhada, a totalidade do trabalho desenvolvido ao longo do projeto de Iniciação Científica “IA UFOP”, entre fevereiro e agosto de 2026. O documento tem caráter técnico-descritivo diferente do artigo científico produzido em paralelo, que segue formato e extensão adequados à submissão editorial, este relatório registra o processo completo de desenvolvimento: decisões tomadas, mudanças de rumo, erros encontrados e corrigidos, resultados obtidos e limitações que permanecem em aberto.

O relatório está organizado cronologicamente, acompanhando a evolução real do projeto: da configuração inicial de ambiente à coleta de dados, construção da base de conhecimento vetorial, seleção e ajuste fino de modelos de linguagem, tentativas de implantação local, e elaboração do artigo científico final. Cada seção documenta tanto os êxitos quanto às falhas técnicas encontradas, com a intenção explícita de preservar o histórico completo do processo de pesquisa.

## 1. Contexto e Evolução do Escopo do Projeto

O projeto originou-se de uma proposta de iniciação científica voltada ao ajuste fino (fine-tuning) do modelo QWEN para automação de tarefas administrativas rotineiras da administração pública, com objetivo de reduzir o tempo de execução de processos e aumentar a conformidade normativa, conforme descrito no documento de proposta original do projeto.

Em um planejamento discente subsequente, o orientador Thiago Fontes Santos sugeriu a substituição do modelo-alvo pelo GPT-OSS 120B, modelo de arquitetura Mixture-of-Experts (aproximadamente 117 bilhões de parâmetros totais, 5,1 bilhões ativos por inferência), licenciado sob Apache 2.0 e otimizado para raciocínio e uso de ferramentas. Esse planejamento previa um cronograma de 12 meses, da coleta de dados à publicação de artigo científico e criação de um site de perfil do modelo.

Ao longo da execução prática do projeto, verificou-se que o GPT-OSS 120B era inviável para a infraestrutura computacional disponível (detalhado na Seção 5.1). O escopo do projeto foi então adaptado progressivamente, culminando na arquitetura efetivamente implementada: um sistema de recuperação aumentada por geração (RAG), com o modelo Qwen2.5 como motor de linguagem. Essa adaptação de escopo de um modelo de 120 bilhões de parâmetros para a família Qwen2.5 foi uma decisão técnica motivada por restrições reais de hardware, e não um desvio do planejamento original, mas sim sua viabilização prática.

## 2. Ambientes Computacionais Utilizados

O desenvolvimento do projeto foi conduzido em três ambientes computacionais distintos, cada um atendendo a uma fase específica do trabalho:

- Notebook pessoal Acer Nitro 5 (processador AMD Ryzen 7 3750H, 24 GB de memória RAM, placa de vídeo NVIDIA GeForce GTX 1650 com 4 GB de VRAM) utilizado na fase inicial de prototipagem de scripts, por permitir desenvolvimento ágil sem ocupar espaço físico em laboratório.
- Workstation do Laboratório GOAL (Grupo de Otimização e Algoritmos) (processador Intel i7-4790K, 24 GB de memória RAM, 2 TB de armazenamento, sem placa de vídeo dedicada de alto desempenho) cujo acesso foi negociado institucionalmente para viabilizar armazenamento de checkpoints de treinamento superiores a 15 GB e testes de inferência local.
- Google Colaboratory (GPU Tesla T4, 16 GB de VRAM, ambiente efêmero com armazenamento persistente via Google Drive) utilizado nas etapas de ajuste fino do modelo e nos testes de validação final do sistema.

A convivência entre um notebook pessoal com VRAM limitada e uma workstation de laboratório sem GPU dedicada tornou-se, ao longo do projeto, não apenas uma limitação prática, mas também um objeto de estudo em si o projeto documentou empiricamente os gargalos de hardware típicos de equipamentos de baixo desempenho disponíveis em setores administrativos de instituições públicas (Seção 6).

### 3. Etapa 1 — Coleta de Dados

A coleta de documentos oficiais foi implementada por meio de um script em Python que automatizou o acesso ao portal público de atos normativos da UFOP (Sistema de Órgãos Colegiados SOC UFOP).

#### 3.1 Funcionamento do Scraper

O script percorreu sequencialmente um intervalo de 5.001 números de resolução (de 8137 a 13137), requisitando a página HTML correspondente a cada número. Para cada documento, duas verificações principais eram realizadas antes do download:

- Filtro de documentos revogados: por meio de expressão regular, o script identificava a presença do marcador “(REVOGADA)” no corpo da página e descartava automaticamente esses documentos, evitando a inclusão de normas sem validade vigente na base de conhecimento.
- Extração do link do PDF: uma segunda expressão regular localizava o caminho do arquivo PDF embutido no HTML da página de resolução, que era então baixado e salvo com nomenclatura padronizada pelo número da resolução, preservando a rastreabilidade cronológica do acervo.

Um intervalo de um segundo entre requisições consecutivas foi adotado como medida de uso responsável do servidor do portal, resultando em um tempo total de coleta superior a uma hora apenas em tempo de espera entre requisições.

### 3.2 Colaboração Externa

O desenvolvimento inicial do script de coleta contou com colaboração decisiva do colega Guilherme Augusto Anício Drummond do Nascimento. Essa colaboração também ampliou a perspectiva do discente sobre o uso da linguagem Python como ferramenta de automação de uso cotidiano, e não apenas em contextos pontuais de disciplina.

### 3.3 Resultados Quantitativos e Perdas de Dados

Do total de 5.001 números de resolução verificados, o processo de coleta e conversão resultou nos seguintes números, documentados de forma explícita para fins de transparência metodológica:

| Etapa                                                         | Quantidade |
|---------------------------------------------------------------|------------|
| Números de resolução verificados                              | 5.001      |
| Arquivos PDF efetivamente recuperados e convertidos           | 4.700      |
| Documentos de alta integridade (após descarte de corrompidos) | 3.700      |
| Segmentos de texto (chunks) indexados na base vetorial final  | 10.314     |

A diferença entre os 5.001 números verificados e os 4.700 PDFs recuperados corresponde, em parte, a documentos revogados descartados automaticamente pelo filtro, e em parte a falhas pontuais de requisição (erros de conexão ou páginas sem link de PDF localizável). A diferença entre os 4.700 PDFs convertidos e os 3.700 documentos de alta integridade corresponde ao descarte de arquivos corrompidos ou com falhas de extração de texto, uma taxa de perda de aproximadamente 21% nessa etapa, não quantificada em detalhe por tipo de falha, o que é registrado aqui como uma limitação da documentação do processo.

### 3.4 Limitações Identificadas na Etapa 1

- Ausência de mecanismo de nova tentativa (retry): falhas de conexão pontuais resultam na perda definitiva do documento correspondente, sem reprocessamento automático.
- Ausência de verificação de duplicidade: reexecuções do script no mesmo intervalo numérico resultariam em novo download dos arquivos já obtidos, sobrescrevendo-os de forma seguro, porém consumindo tempo e banda desnecessariamente.
- Taxa de descarte de aproximadamente 21% entre PDFs convertidos e documentos de alta integridade, sem categorização detalhada das causas de corrupção ou falha de extração.

## 4. Etapa 2 — Construção da Base de Conhecimento Vetorial (RAG)

A segunda etapa do projeto transformou o acervo de documentos brutos coletados em uma base de conhecimento vetorial pesquisável semanticamente, implementada como notebook de cinco células no Google Colab.

### 4.1 Descoberta Crítica: Dataset Anterior com Respostas Vazias

Durante esta etapa, identificou-se que um dataset anterior do projeto (“dataset\_ufop.jsonl”), usado em uma tentativa de treinamento prévia e não documentada em detalhe, apresentava o campo “output” vazio em todos os seus registros. Essa descoberta forneceu uma explicação técnica retrospectiva para o mau desempenho de um modelo treinado anteriormente: como o ajuste fino supervisionado calcula a função de perda sobre o texto-alvo (“output”), um conjunto de treinamento com esse campo sistematicamente vazio ensina o modelo a gerar respostas vazias ou truncadas, e não a responder de fato às perguntas. Essa foi tratada como uma lição metodológica relevante, corrigida nas etapas seguintes por meio da geração automática de respostas de referência a partir do próprio texto dos documentos (Seção 5).

## 4.2 Extração e Limpeza de Texto

A extração de texto dos PDFs adotou uma estratégia de dupla biblioteca: extração primária com PyMuPDF, com fallback para pdfplumber nos casos em que a primeira não produzia texto satisfatório (tipicamente documentos com tabelas ou colunas múltiplas). O texto extraído foi submetido a limpeza para remoção de ruídos característicos do sistema de tramitação eletrônica institucional: timestamps, URLs do SEI/UFOP, cabeçalhos repetidos e blocos de assinatura digital.

## 4.3 Segmentação e Indexação

O texto de cada documento foi segmentado em blocos de aproximadamente 400 palavras, com sobreposição de 60 palavras entre blocos consecutivos, para preservar o contexto de trechos localizados na fronteira entre dois segmentos. Cada segmento foi convertido em vetor por meio do modelo multilíngue intfloat/multilingual-e5-base, com os prefixos obrigatórios “passage:” (indexação) e “query:” (busca) — detalhe técnico do modelo cuja omissão degradaria a qualidade da recuperação. Os vetores foram indexados em um banco ChromaDB persistente, configurado para similaridade de cosseno, em lotes de 64 itens, com mecanismo de checkpoint para retomada em caso de desconexão do Colab.

## 4.4 Resultado Final e Validação

Ao final da Etapa 2, a base de conhecimento vetorial continha 10.314 segmentos de texto indexados. Testes exploratórios de recuperação, com perguntas típicas do contexto de iniciação científica institucional (por exemplo, sobre requisitos do PIBIC e prazos de relatórios), confirmaram que o sistema recuperava trechos tematicamente pertinentes às consultas, com valores de similaridade de cosseno compatíveis com respostas relevantes.

## 4.5 Falhas Cosméticas Identificadas

- A mensagem de conclusão exibida ao final do notebook da Etapa 2 informava incorretamente “ETAPA 1 CONCLUÍDA!”, um resquício de nomenclatura interna de versão anterior do notebook, sem efeito sobre a execução, mas identificado como pendência de correção na documentação do código.

## 5. Etapa 3 — Seleção de Arquitetura e Ajuste Fino do Modelo

### 5.1 Tentativas com Modelos de Maior Porte (GPT-OSS)

Antes da consolidação da arquitetura final, avaliou-se a viabilidade de dois modelos de maior porte sugeridos no planejamento inicial do projeto:

- **GPT-OSS 120B:** a tentativa de carregamento resultou em erro de estouro de memória (Out of Memory) imediato, uma vez que o modelo de arquitetura Mixture-of-Experts com aproximadamente 117 bilhões de parâmetros totais exige mais de 200 GB de memória de vídeo para operação, valor absolutamente incompatível com qualquer um dos ambientes computacionais disponíveis ao projeto.
- **GPT-OSS 20B:** modelo tecnicamente viável de carregar, sobre o qual foram conduzidos treinamentos de pré-treinamento contínuo (continuous pre-training) e de ajuste fino supervisionado (SFT). O modelo resultante, no entanto, apresentou latência elevada de resposta e baixa precisão na geração de texto em língua portuguesa, o que motivou sua descontinuação como modelo-base do projeto.

A partir dessas duas tentativas malsucedidas, a decisão técnica foi migrar para a família de modelos Qwen2.5, que demonstrou equilíbrio mais adequado entre porte do modelo, velocidade de inferência e capacidade de compreensão do português no domínio normativo da UFOP.

### 5.2 Ajuste Fino do Qwen2.5-1.5B-Instruct via QLoRA

A primeira variante da família Qwen efetivamente ajustada foi o Qwen2.5-1.5B-Instruct, por meio da técnica QLoRA (adaptação de baixo posto com quantização de 4 bits). O conjunto de treinamento foi construído a partir dos segmentos de texto extraídos, formatados no padrão ChatML, com respostas de referência geradas automaticamente por meio de extração por expressão regular de elementos identificáveis no texto (número da resolução, data de publicação, artigos normativos) — abordagem que corrigiu o problema de respostas vazias identificado na Seção 4.1.

Configuração técnica do ajuste fino:

- Quantização: NF4 (NormalFloat4) de 4 bits, com quantização dupla (double quantization) e cálculo em precisão float16.
- LoRA: rank 16, fator de escala (alpha) 32, aplicado às camadas de projeção de atenção (q\_proj, k\_proj, v\_proj, o\_proj) e às camadas feed-forward (gate\_proj, up\_proj, down\_proj), dropout de 0,05.
- Treinamento: 3 épocas, tamanho de lote unitário com acumulação de gradiente equivalente a lote efetivo de 8 amostras, otimizador paged\_adamw\_8bit, taxa de aprendizado inicial de  $2 \times 10^{-4}$  com escalonamento por cosseno e 50 passos de aquecimento.

- Persistência: callback customizado (DriveBackupCallback) copiando cada checkpoint salvo diretamente para o Google Drive, essencial dada a instabilidade de sessões gratuitas do Colab.

### 5.3 Código Anterior Descontinuado (Unsloth + SFTTrainer)

Registra-se, para fidelidade histórica, que uma versão anterior do pipeline de ajuste fino utilizava a biblioteca Unsloth (FastLanguageModel) em conjunto com o SFTTrainer da biblioteca TRL, conforme documentado no relatório técnico de fevereiro–abril de 2026 (que foi descontinuado). Essa abordagem foi posteriormente substituída pela implementação final, que utiliza a classe Trainer padrão da biblioteca Transformers com uma classe de dataset customizada (UFOPDataset) e mascaramento manual de labels. Ambas as abordagens são tecnicamente equivalentes em resultado, mas o código efetivamente utilizado na versão final do projeto é o segundo, não o primeiro.

```
# Trecho ilustrativo do código de ajuste fino final (não-Unsloth):
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True, bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True, bnb_4bit_compute_dtype=torch.float16)
lora_config = LoraConfig(r=16, lora_alpha=32,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj"],
    lora_dropout=0.05, bias="none", task_type="CAUSAL_LM")
```

### 5.4 Resultado do Ajuste Fino em 1,5 Bilhão de Parâmetros: Alucinação

O treinamento do Qwen2.5-1.5B-Instruct foi concluído com sucesso do ponto de vista técnico, produzindo um adaptador LoRA funcional (“modelo\_ufop\_adapter\_v2”). Testes qualitativos, no entanto, revelaram que o modelo mesmo operando em conjunto com a base de recuperação (RAG) produzia respostas parcialmente desalinhadas com o conteúdo dos documentos fornecidos como contexto, fenômeno identificado como alucinação.

### 5.5 Migração para o Qwen2.5-7B-Instruct

Diante da alucinação observada, a decisão técnica foi substituir o modelo de base pela variante Qwen2.5-7B-Instruct, combinada à mesma base de recuperação, sem a aplicação de ajuste fino adicional nessa variante maior. Testes qualitativos indicaram melhora perceptível na fidelidade das respostas ao contexto fornecido, sem os sinais de alucinação observados na variante de 1,5 bilhão de parâmetros.

### 5.6 Achado Crítico: Inexistência de Adaptador Treinado para o Modelo de 7B

Durante os testes de implantação em ambiente Google Colab (detalhados na Seção 6.4), foi identificado por meio de erro de incompatibilidade de dimensões (“size mismatch”) ao tentar aplicar o adaptador salvo sobre o modelo de 7 bilhões de parâmetros que nenhum adaptador LoRA foi efetivamente treinado para a variante Qwen2.5-7B-Instruct. A inspeção do arquivo



“adapter\_config.json” confirmou que o único adaptador salvo (“modelo\_ufop\_adapter\_v2”) foi treinado exclusivamente sobre o modelo Qwen2.5-1.5B-Instruct.

A justificativa fornecida para essa decisão, no momento em que foi tomada, foi de que “o que importava era o RAG treinado” ou seja, a hipótese de que a base de recuperação vetorial, combinada a um modelo-base suficientemente capaz, seria suficiente para produzir respostas fiéis aos documentos, independentemente de ajuste fino adicional. Essa hipótese é tecnicamente defensável e corresponde a uma arquitetura de RAG amplamente utilizada na prática. Entretanto, ela representa um desvio não formalizado em relação aos objetivos específicos originais do projeto, que previam avaliação comparativa formal (precisão, revocação, F1, BLEU, avaliação humana) entre o modelo com e sem ajuste fino comparação que, sem um adaptador de 7B, não pôde ser realizada. Este ponto permanece como a principal lacuna metodológica em aberto ao final do projeto, registrada tanto neste relatório quanto no artigo científico produzido.

## **6. Implantação Local e Testes de Hardware**

### **6.1 Pacote de Implantação Standalone**

Foi desenvolvido um pacote de implantação (“Assistente\_UFOP”), contendo um script principal (“ufop\_ia.py”), um executável de inicialização (“itiniar.bat”), a base vetorial ChromaDB e a pasta do adaptador treinado, com o objetivo de permitir que o sistema fosse testado em computadores de terceiros, incluindo o orientador do projeto.

### **6.2 Falha Original Identificada e Corrigida: Adaptador Nunca Aplicado**

Na primeira versão revisada do script de implantação, identificou-se uma falha significativa: o código carregava apenas o modelo base (Qwen2.5-7B-Instruct) via AutoModelForCausalLM, sem nenhuma chamada a PeftModel.from\_pretrained para aplicar o adaptador LoRA. Isso significava que, tecnicamente, o sistema em produção estava servindo o modelo base “cru” combinado ao RAG, sem nenhum dos ganhos do ajuste fino realizado falha corrigida com a inclusão explícita do carregamento e aplicação do adaptador.

### **6.3 Teste na GTX 1650 (4 GB de VRAM): Falha por Esgotamento de Memória**

A primeira tentativa de execução local do modelo de 7 bilhões de parâmetros em 4 bits, na GTX 1650 do notebook pessoal, falhou por esgotamento de memória de vídeo (CUDA Out of Memory), interrompendo o carregamento dos pesos do modelo em aproximadamente 24% de progresso. O diagnóstico técnico realizado estimou a necessidade real de VRAM entre 5,5 e 6,5 GB considerando os pesos quantizados (~3,8-4 GB), a camada de embeddings em precisão de 16 bits (~1 GB) e o overhead de contexto CUDA (~300-600 MB) valor superior aos 4,3 GB disponíveis na placa.

## 6.4 Teste na Workstation do Lab GOAL: Latência Extrema em CPU

Um segundo caso de implantação, na workstation do Laboratório GOAL (processador Intel i7-4790K, sem GPU dedicada de alto desempenho), evidenciou de forma ainda mais acentuada as limitações de hardware de baixo desempenho: a execução do modelo em precisão de 16 bits inteiramente sobre a CPU resultou em tempo de resposta superior a 30 minutos por consulta, mesmo após tentativas de mitigação como processamento paralelo entre os 8 núcleos do processador (multithreading) e penalização de repetição de tokens (repetition\_penalty).

## 6.5 Validação em Ambiente Google Colab (Tesla T4)

Testes de validação foram conduzidos em ambiente Google Colab, cuja GPU Tesla T4 (16 GB de VRAM) comporta com folga o modelo de 7 bilhões de parâmetros em 4 bits. Durante esses testes, foram identificados e corrigidos os seguintes problemas técnicos pontuais, documentados aqui por completude:

- Erro “FileNotFoundError: nvidia-smi”: ocorreu quando o ambiente de execução do Colab não estava configurado para uso de GPU (configuração padrão é CPU); corrigido orientando a troca do acelerador de hardware para GPU T4 e tornando a verificação resiliente a essa ausência.
- Erro “NameError: PASTA\_ADAPTER not defined”: ocorreu quando células do notebook foram executadas fora de ordem, ou quando a sessão do Colab reiniciou silenciosamente entre a execução de duas células; corrigido tornando cada célula independente, redefinindo os caminhos necessários sem depender do estado de execução de células anteriores.
- Erro de incompatibilidade de dimensões (“size mismatch”) ao aplicar o adaptador sobre o modelo de 7B descrito em detalhe na Seção 5.6 que confirmou a inexistência de um adaptador treinado especificamente para essa variante do modelo.

Após a correção dos problemas de ambiente, a base de conhecimento vetorial foi carregada com sucesso (10.314 documentos indexados) e o modelo base Qwen2.5-7B-Instruct em 4 bits foi carregado integralmente na GPU T4, confirmando a viabilidade técnica da arquitetura RAG combinada ao modelo base em hardware com VRAM suficiente.

## 7. Estratégia de Implantação em Hardware Limitado (GGUF / llama.cpp)

Como resposta direta às limitações de hardware documentadas na Seção 6, foi desenhada uma estratégia alternativa de implantação baseada no formato de quantização GGUF, executado por meio da biblioteca llama.cpp e de seus bindings Python (llama-cpp-python). Essa estratégia permite particionar as camadas do modelo entre a memória de vídeo disponível e a memória RAM do sistema, por meio do parâmetro configurável `n_gpu_layers`, possibilitando a execução do modelo completo mesmo em placas de vídeo com memória insuficiente para comportar a totalidade dos parâmetros quantizados.

O pipeline projetado compreende três etapas: (1) mesclagem do adaptador LoRA ao modelo base por meio de `merge_and_unload()`, executável em CPU; (2) conversão do modelo mesclado para o formato GGUF em precisão F16, seguida de quantização para Q4\_K\_M (tamanho final estimado entre 4,5 e 5 GB); e (3) execução via llama-cpp-python, com ajuste manual do número de camadas alocadas à GPU conforme a VRAM disponível.

É importante registrar, com transparência, que essa estratégia foi integralmente projetada e documentada, mas não pôde ser empiricamente validada até o encerramento do projeto, uma vez que sua primeira etapa (mesclagem do adaptador) depende da existência de um adaptador LoRA treinado para o modelo Qwen2.5-7B-Instruct o qual, conforme registrado na Seção 5.6, nunca chegou a ser treinado. A estratégia GGUF permanece, portanto, como uma contribuição de engenharia pronta para execução, condicionada à decisão de continuidade do projeto.

## **8. Elaboração do Artigo Científico**

Em paralelo à consolidação técnica do projeto, foi produzido um artigo científico completo, adaptado ao template da Revista Educação & Ensino (Centro Universitário Ateneu — UniAteneu), estruturado em Resumo/Abstract, Introdução, Referencial Teórico (três subseções), Metodologia (quatro subseções), Resultados e Discussão, Conclusão, Agradecimentos e Referências.

### **8.1 Revisão e Correção do Referencial Bibliográfico**

Durante a elaboração do artigo, identificou-se uma inconsistência na lista de referências do documento de proposta original do projeto: ao menos uma referência (relativa à família de modelos SaulLM) apresentava atribuição de autoria não correspondente aos autores reais desse trabalho, de acordo com o conhecimento verificável disponível. Diante desse achado, duas referências de atribuição não verificável foram removidas e substituídas por oito referências reais e diretamente verificáveis os artigos fundacionais das próprias técnicas empregadas no projeto (LoRA, QLoRA, geração aumentada por recuperação, entre outras), identificadas a partir da lista de referências do relatório técnico de fevereiro–abril de 2026. Buscou-se ainda, conforme orientação do próprio template da revista, um artigo já publicado no periódico com temática correlata, localizado e devidamente referenciado.

### **8.2 Seção de Agradecimentos**

Foi incluída seção de agradecimentos nominal a Guilherme Augusto Anício Drummond do Nascimento, pela contribuição técnica na etapa de coleta de dados, e a Brendha Alexania Pereira Godinho, pelo apoio pessoal e disponibilização de equipamento computacional durante o desenvolvimento do projeto.

## 9. Síntese Consolidada de Falhas e Obstáculos Técnicos

Esta seção reúne, de forma consolidada, a totalidade das falhas, erros e obstáculos técnicos documentados ao longo do projeto, independentemente da seção em que foram originalmente descritos, para fins de registro completo.

- GPT-OSS 120B inviável: exige mais de 200 GB de VRAM, incompatível com qualquer ambiente disponível.
- GPT-OSS 20B: após CPT e SFT, apresentou latência elevada e baixa precisão em português.
- Dataset anterior (dataset\_ufop.jsonl) com campo “output” vazio em todos os registros, causando treinamento de um modelo “mudo” em tentativa não documentada em detalhe.
- Qwen2.5-1.5B-Instruct ajustado via QLoRA apresentou alucinação mesmo operando com RAG.
- Função de fallback “formatar\_do\_chunk” (usada apenas na ausência do dataset original) copiava o texto de entrada como resposta de saída, o que poderia reforçar comportamento de eco caso esse caminho tivesse sido efetivamente exercitado no treinamento final.
- Nenhum adaptador LoRA foi treinado para o modelo Qwen2.5-7B-Instruct, impossibilitando a comparação formal com/sem ajuste fino prevista nos objetivos originais do projeto.
- Script de implantação original carregava o modelo base sem aplicar o adaptador treinado — falha silenciosa que resultaria em uso do modelo sem nenhum ganho do ajuste fino, corrigida durante a revisão do código.
- GTX 1650 (4 GB de VRAM): falha por esgotamento de memória ao carregar o modelo de 7B em 4 bits.
- Workstation do Lab GOAL (CPU pura): tempo de resposta superior a 30 minutos por consulta.
- Múltiplos erros de ambiente no Google Colab: GPU não habilitada por padrão, variáveis perdidas entre células por reinício de sessão, e o erro de incompatibilidade de dimensões do adaptador.
- Mensagem de conclusão incorreta no notebook da Etapa 2 (“ETAPA 1 CONCLUÍDA”), resquício de nomenclatura de versão anterior.
- Inconsistência de nomenclatura entre células do notebook de ajuste fino (título mencionando “Qwen2.5-7B” em célula que na verdade carregava o Qwen2.5-1.5B; título mencionando “SFTTrainer” em célula que na verdade usa a classe Trainer padrão).
- Inconsistência de nomes de pasta do adaptador entre células do mesmo notebook (“modelo\_ufop\_adapter\_v1\_5b” definida e nunca usada; “modelo\_ufop\_adapter\_v2” efetivamente utilizada).

- Referência bibliográfica com atribuição de autoria não verificável na proposta original do projeto (família SaulLM), corrigida na elaboração do artigo científico.
- Estratégia de implantação via GGUF/llama.cpp projetada, porém não empiricamente validada até o encerramento do projeto, por depender de um adaptador de 7B inexistente.
- Ausência de avaliação quantitativa formal (precisão, revocação, F1, BLEU, avaliação humana) prevista nos objetivos específicos originais do projeto — não realizada até o encerramento.

## 10. Síntese Consolidada de Conquistas

- Scraper funcional e responsável, com filtro automático de documentos revogados, coletando o acervo-alvo de aproximadamente 5.000 documentos oficiais da UFOP.
- Pipeline de ETL robusto, com estratégia de extração em dupla biblioteca (PyMuPDF + pdfplumber) e segmentação com sobreposição para preservação de contexto.
- Base de conhecimento vetorial funcional, com 10.314 segmentos indexados e recuperação semântica validada qualitativamente.
- Diagnóstico correto e documentado da causa-raiz de uma falha de treinamento anterior (dataset com respostas vazias).
- Comparação empírica, ainda que não formalmente quantificada, entre três escalas de modelo (120B inviável, 20B com baixo desempenho, 7B viável), com registro das razões técnicas de descarte de cada alternativa.
- Pipeline de ajuste fino QLoRA funcional e comprovado (na variante de 1,5B), tecnicamente transferível à variante de 7B.
- Diagnóstico preciso de gargalos de hardware em dois ambientes reais e independentes (VRAM insuficiente em GPU de entrada; latência extrema em CPU antiga), com estimativas quantitativas de memória necessária.
- Desenho completo de uma estratégia de implantação híbrida GPU/CPU (GGUF) diretamente alinhada ao objetivo do projeto de viabilizar IA em hardware de baixo desempenho.
- Correção de falha crítica de implantação (adaptador nunca aplicado) antes do envio do sistema para teste externo.
- Identificação e correção de uma inconsistência de atribuição bibliográfica na proposta original do projeto, fortalecendo o rigor acadêmico do artigo final.
- Produção de um artigo científico completo, formatado e pronto para submissão, com referencial teórico fundamentado em fontes primárias verificáveis.
- Documentação extensa e reproduzível de todo o processo, incluindo esta síntese final.

## 11. Estado Final do Projeto e Recomendações para Eventual Continuidade

Ao encerramento deste ciclo de Iniciação Científica, o projeto entrega um sistema RAG funcional base vetorial de 10.314 documentos indexados combinada ao modelo Qwen2.5-7B-Instruct validado qualitativamente quanto à ausência de alucinação, além de um artigo científico completo e um conjunto de scripts documentados para coleta, indexação, ajuste fino e implantação.

A principal lacuna em aberto é a ausência de um adaptador LoRA treinado especificamente para o modelo de 7 bilhões de parâmetros, o que impede tanto a avaliação quantitativa formal prevista nos objetivos originais quanto a validação empírica da estratégia de implantação em GGUF já projetada. Caso haja continuidade do projeto, por este ou outro discente, os seguintes passos são recomendados, em ordem de prioridade:

- Executar o ajuste fino QLoRA sobre o Qwen2.5-7B-Instruct, reaproveitando o pipeline já validado na variante de 1,5B.
- Conduzir avaliação quantitativa formal (precisão, revocação, F1 na extração de entidades; BLEU e avaliação humana na geração de texto), comparando explicitamente as configurações com e sem ajuste fino.
- Executar e validar empiricamente o pipeline de conversão GGUF já documentado, mensurando tempo de resposta em função do número de camadas alocadas à GPU em hardware de baixo desempenho.
- Investigar e quantificar as causas específicas da perda de aproximadamente 21% dos documentos entre a conversão de PDF e o descarte por corrupção, na Etapa 1.
- Implementar mecanismo de nova tentativa (retry) e verificação de duplicidade no script de coleta de dados.

Este relatório, em conjunto com o artigo científico produzido e os scripts documentados, constitui o registro completo e fidedigno do trabalho realizado entre fevereiro e agosto de 2026 no âmbito do projeto IA UFOP.